

NOIP 模拟赛

By Yazid Wong

第一试

题目名称	队栈	副本挑战	网络连接
题目类型	传统型	传统型	传统型
目录	stqueue	instance	network
可执行文件名	stqueue	instance	network
输入文件名	stqueue.in	instance.in	network.in
输出文件名	stqueue.out	instance.out	network.out
每个测试点时限	1.0 秒	2.0 秒	2.0 秒
内存限制	512 MB	512 MB	512 MB
测试点/包数目	10	25	20
测试点是否等分	是	是	是

提交源程序文件名

对于 C++ 语言	stqueue.cpp	instance.cpp	network.cpp
对于 C 语言	stqueue.c	instance.c	network.c
对于 Pascal 语言	stqueue.pas	instance.pas	network.pas

编译选项

对于 C++ 语言	-lm
对于 C 语言	-lm
对于 Pascal 语言	

队栈 (staqueue)

【题目描述】

Yazid 是一名 OI 初学者。他最近在研究基础数据结构：队列和栈。某天，Yazid 脑洞大开，发明了一种叫做“队栈”的数据结构。

众所周知，队列是先进先出的数据结构，而栈是先进后出的数据结构。而 Yazid 发明的队栈则同时支持查询并删除其中最早被插入的元素和最晚被插入的元素。

Yazid 有一个初始为空的队栈和一个黑盒（这是一个存放数字的盒子），接下来，他要依次执行 Q 个操作。操作分为 4 种：

1. **push**：将一个非负整数加入队栈。
2. **pop_queue**：找出队栈中最早被插入的元素，将其取出放入黑盒，并从队栈中删除。
3. **pop_stack**：找出队栈中最晚被插入的元素，将其取出放入黑盒，并从队栈中删除。
4. **print**：将黑盒中的所有数取出，并按被放入的先后顺序从左到右排列得到一个整数。

- 例如：黑盒中依次被放入了 0,23,330,6，那么获得并打印的整数即是 233306。

由于 Yazid 懒于在每次 **push** 操作时想插入的数，因此他提前写好了一个长度为 n 的插入序列 A （下标从 1 开始）。在接下来的所有 **push** 操作中，Yazid 会依次地、循环地将这些数加入队栈。具体来说：

- 在首次 **push** 操作时，Yazid 会将 A_1 加入队栈。
 - 在之后的每次 **push** 操作中，假设 Yazid 上次 **push** 时加入的数是 A_i ，则本次他会将 A_{i+1} 加入队栈。
 - 在上一种情况中，特别地，如果 $i = n$ ，则 Yazid 本次会将 A_1 加入队栈。
- 请你依次输出 Yazid 通过 **print** 操作获得的整数。

【输入格式】

从文件 **staqueue.in** 中读入数据。

第 1 行一个整数 n ，表示插入序列的长度。

第 2 行 n 个用单个空格隔开的非负整数 $A_1, A_2, \dots, A_n$ ，描述插入序列。

第 3 行一个整数 Q ，表示操作数目。

第 4 行 Q 个紧挨着的 1 ~ 4 之间的数字，依次描述每个操作，每个数字表示操作的编号（各操作对应的编号见【题目描述】）。

- 保证所有操作的合法性。即保证：

- 执行任意 **pop_queue** 和 **pop_stack** 操作时队栈不为空。

– 执行任意 `print` 操作时黑盒不为空。

【输出格式】

输出到文件 `staqueue.out` 中。

对于所有 `print` 操作，输出一行一个整数，表示该 `print` 操作中 Yazid 获得的数。

【样例 1 输入】

```
2
1 2
7
1112324
```

【样例 1 输出】

```
112
```

【样例 2 输入】

```
4
0 23 330 6
12
121313124134
```

【样例 2 输出】

```
233306
0
```

【样例 3】

见选手目录下的 `staqueue/staqueue3.in` 与 `staqueue/staqueue3.ans`。

【子任务】

测试点编号	$n =$	$Q =$	其他约定
1	1	0	无
2		10	
3		1000	
4~5	10^4	10^5	保证不存在操作 2
6~7			保证不存在操作 3
8~9			保证 $A_i > 0$
10		10^7	无

对于所有测试点，保证 $n \leq 10^4$ ， $A_i < 1000$ ， $Q \leq 10^7$ 。

副本挑战 (instance)

【题目描述】

Yazid 最近沉迷于一款冒险类游戏无法自拔。

这款游戏的地图可以抽象成一张包含 n 个节点 m 条边的无向图（节点和边的编号均从 1 开始），每条边带有边权，也就是说通过每条边所需的时间不尽相同。

Yazid 接到了 k 个副本任务，每个任务都要求 Yazid 进入一个独一无二的副本挑战 Boss。

其中，第 i 个任务 ($1 \leq i \leq k$) 的副本入口在节点 u_i ，其入口开放时间为 $[l_i, r_i]$ ，换言之，Yazid 需要在规定时间内到达（或等候在）节点 u_i ，才能进入副本，挑战 Boss。

除此之外，每个副本的难度对于 Yazid 而言也不尽相同，经过精确的计算，Yazid 成功挑战第 i 个副本所需的时间为 t_i ，也就是说，如果 Yazid 在 T 时刻进入副本 i ，那么他将在 $T + t_i$ 时刻将 Boss 挑落马下，并继续他的征程。

需要注意的是，入口开放时间只对玩家的进入副本时间作出限制，玩家仅仅被要求在入口关闭前进入副本，并不被要求在入口关闭前完成 Boss 挑战。

每个任务的重要程度也是不尽相同的。完成第 i 个任务可以获得 c_i 个金币的奖励。

在第 0 时刻，Yazid 的角色身处节点 1。

请问，假设 Yazid 的策略安排得当，他总共最多能够获得多少金币奖励呢？

【输入格式】

从文件 `instance.in` 中读入数据。

第 1 行 2 个用单个用单个空格隔开的正整数 n, m ，分别表示图的节点数、边数。

接下来 m 行，每行 3 个用单个空格隔开的整数 x, y, z ，描述一条连接节点 x, y 的、需要花 z 单位时间通过的无向边。

- 在这里，我们保证 $1 \leq x, y \leq n$ 。

接下来 1 行 1 个正整数 k ，表示副本任务的数量。

接下来 k 行，每行 5 用单个空格隔开的整数 u_i, l_i, r_i, t_i, c_i 描述一个任务，意义见

【题目描述】。

- 在这里，我们保证 $1 \leq u_i \leq n, l_i \leq r_i$ 。

【输出格式】

输出到文件 `instance.out` 中。

输出一行一个整数，表示 Yazid 最多能够获得的奖励金币数目。

【样例 1 输入】

```
3 2
1 2 10
2 3 10
2
1 0 50 20 1
2 0 39 20 1
```

【样例 1 输出】

```
2
```

【样例 1 解释】

Yazid 应先前往 2 号节点执行第 2 个任务，然后再回到 1 号节点执行第 1 个任务，才能同时完成 2 个任务。

【样例 2 输入】

```
4 6
1 2 1
1 3 5
1 4 10
2 3 2
2 4 3
3 4 15
3
1 0 20 2 1
3 4 20 6 2
4 15 15 2 2
```

【样例 2 输出】

```
4
```

【样例 2 解释】

Yazid 可以依次完成任务 2,3，但如若这么做，他将不得不错过任务 1，因此他最多只能获得 4 个金币。

【样例 3】

见选手目录下的 *instance/instance3.in* 与 *instance/instance3.ans*。

【子任务】

测试点编号	$n =$	$m =$	$k =$	特殊约定
1~2	1	0	1	无
3~6			9	
7~9			18	
10			18	
11	200	20000	1	无
12			2	
13			3	
14~19			9	
20~25			18	

对于所有测试点，保证 $n \leq 200$, $m \leq 20,000$, $k \leq 18$, $1 \leq z \leq 10^4$, $0 \leq l_i, r_i, t_i \leq 10^7$, $c_i \leq 10^4$ 。

网络连接 (network)

【题目描述】

Yazid 所在的 Duck 公司最近面临了一个棘手的网络连接问题。

Duck 公司共有 n 台电脑，电脑之间可以通过网线相连。网络管理员 Yazid 使用了 $n - 1$ 根网线连接这些电脑，其中每跟网线都连接了两台电脑。所有的电脑都由这些网线直接或间接互联，根据常识， $n - 1$ 也是最少的需要使用的网线数目。

近期，由于公司拓展业务，部分电脑需要被分离出原网络，建立起新的局域网。具体来说，每台电脑均有恰好 50% 的概率被分配于新任务，否则即执行旧任务。所有电脑被分配新、旧业务的概率是相互独立的。

在分配任务后，对于任意网线，如果它连接的两台电脑执行的任务不同，Duck 公司就将会将它们切断。而 Yazid 的任务，即是使用一些新的网线，将执行同一任务的电脑全部互联。

虽然 Duck 公司有足够的新网线，但事实上，最棘手的问题在于紧缺的网口。被切断网线所占据的网口已不能再用，而每台电脑恰好有 1 个额外的网口可用于接入网线，每需增加 1 个网口，Yazid 就需要支付 1 个金币。这也就意味着，如果 Yazid 希望在同一台电脑上新连接超过 1 根网线，他就需要支付额外的费用。

Yazid 聪明绝顶，一旦任务分配确定，他就一定会花费最少的金币完成目标。而 Duck 公司对 Yazid 的期望花费非常感兴趣，并希望你能帮他们计算该期望值。

而聪明的你应该早已发现，这个期望值与 2^n 的乘积一定是一个整数，因此你只打算告诉 Duck 公司该期望值与 2^n 的乘积对 998,244,353 取模的结果。

【输入格式】

从文件 `network.in` 中读入数据。

本题包含多组数据，第一行一个非负整数 T ，表示数据组数。接下来依次描述每组数据，对于每组数据：

- 第一行一个正整数 n ，表示 Duck 公司的电脑数目。
- 接下来 $n - 1$ 行描述所有初始的网线，每行 2 个用单个空格隔开的正整数 u, v 描述一根连接电脑 u, v 的网线。
 - 电脑的编号从 1 开始。
 - 保证 $1 \leq u, v \leq n$ 。
 - 保证所有电脑被这些网线连通。

【输出格式】

输出到文件 `network.out` 中。

对于每组数据：

- 输出一行一个整数，表示期望花费金币数与 2^n 的乘积对 998,244,353 取模的结果。

【样例 1 输入】

```
2
2
1 2
4
1 2
3 1
4 1
```

【样例 1 输出】

```
0
2
```

【样例 1 解释】

对于第一组数据，无论任务如何分配，都一定不需要新的网线了。

对于第二组数据，只有 2,3,4 号电脑被分配的任务相同，且与 1 号电脑被分配的任务不同时，才会需要额外的 1 个网口，其发生的概率为 $\frac{1}{8}$ ，因此答案为 $\frac{1}{8} \times 2^4 \bmod 998,244,353 = 2$ 。

【样例 2 输入】

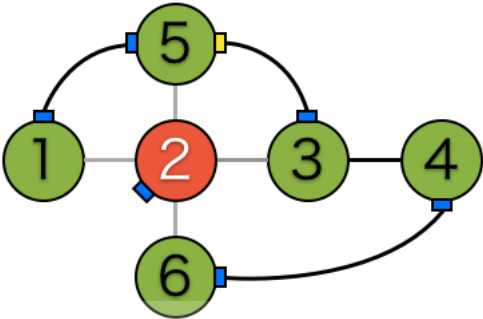
```
1
6
1 2
2 3
3 4
2 5
2 6
```

【样例 2 输出】

```
24
```

【样例 2 解释】

下图是一个例子：如果只有 2 号电脑被分配于新任务，那么聪明绝顶的 Yazid 可以按如下方法做连接（蓝、黄网口分别表示免费、付费网口），使其只需花费 1 个金币。



【样例 3】

见选手目录下的 `network/network3.in` 与 `network/network3.ans`。

【子任务】

测试点编号	n	其他约定
1	≤ 3	无
2	≤ 5	
3	≤ 8	
4~5	≤ 17	
6	≤ 30	对于任意初始网线有 $u = 1$
7		对于任意初始网线有 $u + 1 = v$
8~10		无
11	≤ 100	对于任意初始网线有 $u = 1$
12		对于任意初始网线有 $u + 1 = v$
13	≤ 400	对于任意初始网线有 $u = 1$
14		对于任意初始网线有 $u + 1 = v$
15~17		无
18	≤ 2000	对于任意初始网线有 $u = 1$
19		对于任意初始网线有 $u + 1 = v$
20		无

对于所有测试点，保证 $T \leq 3$ 。

对于所有测试点中的所有数据，保证 $n \leq 2000$ 。